

NLP_Practical_04

April 7, 2026

0.0.1 Step 1: Install & Import

```
[1]: # If not installed
      # !pip install torch

import torch
import torch.nn as nn
import math
```

0.0.2 Step 2: Positional Encoding

```
[2]: class PositionalEncoding(nn.Module):
      def __init__(self, d_model, max_len=100):
          super(PositionalEncoding, self).__init__()

          pe = torch.zeros(max_len, d_model)
          position = torch.arange(0, max_len).unsqueeze(1)

          div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) /
          ↪ d_model))

          pe[:, 0::2] = torch.sin(position * div_term)
          pe[:, 1::2] = torch.cos(position * div_term)

          pe = pe.unsqueeze(0)
          self.register_buffer('pe', pe)

      def forward(self, x):
          return x + self.pe[:, :x.size(1)]
```

0.0.3 Step 3: Multi-Head Attention

```
[4]: class MultiHeadAttention(nn.Module):
      def __init__(self, d_model, num_heads):
          super(MultiHeadAttention, self).__init__()

          self.num_heads = num_heads
          self.d_model = d_model
```

```

        self.head_dim = d_model // num_heads

        self.q_linear = nn.Linear(d_model, d_model)
        self.k_linear = nn.Linear(d_model, d_model)
        self.v_linear = nn.Linear(d_model, d_model)

        self.fc_out = nn.Linear(d_model, d_model)

    def forward(self, query, key, value):
        batch_size = query.shape[0]

        Q = self.q_linear(query)
        K = self.k_linear(key)
        V = self.v_linear(value)

        Q = Q.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1,2)
        K = K.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1,2)
        V = V.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1,2)

        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.head_dim)
        attention = torch.softmax(scores, dim=-1)

        out = torch.matmul(attention, V)

        out = out.transpose(1,2).contiguous().view(batch_size, -1, self.d_model)

        return self.fc_out(out)

```

0.0.4 Step 4: Feed Forward Network

```

[5]: class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff=2048):
        super(FeedForward, self).__init__()

        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        return self.fc2(torch.relu(self.fc1(x)))

```

0.0.5 Step 5: Encoder Layer

```

[6]: class EncoderLayer(nn.Module):
    def __init__(self, d_model, num_heads):
        super(EncoderLayer, self).__init__()

```

```

        self.attention = MultiHeadAttention(d_model, num_heads)
        self.norm1 = nn.LayerNorm(d_model)

        self.ff = FeedForward(d_model)
        self.norm2 = nn.LayerNorm(d_model)

    def forward(self, x):
        attn_out = self.attention(x, x, x)
        x = self.norm1(x + attn_out)

        ff_out = self.ff(x)
        x = self.norm2(x + ff_out)

    return x

```

0.0.6 Step 6: Transformer Encoder

```

[7]: class Transformer(nn.Module):
    def __init__(self, vocab_size, d_model=128, num_heads=4, num_layers=2):
        super(Transformer, self).__init__()

        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoding = PositionalEncoding(d_model)

        self.layers = nn.ModuleList([
            EncoderLayer(d_model, num_heads) for _ in range(num_layers)
        ])

        self.fc_out = nn.Linear(d_model, vocab_size)

    def forward(self, x):
        x = self.embedding(x)
        x = self.pos_encoding(x)

        for layer in self.layers:
            x = layer(x)

        return self.fc_out(x)

```

0.0.7 Step 7: Testing the Model

```

[8]: x = torch.randint(0, 50, (2, 5))

model = Transformer(vocab_size=50)

output = model(x)

```

```
print("Input shape:", x.shape)
print("Output shape:", output.shape)
```

```
Input shape: torch.Size([2, 5])
Output shape: torch.Size([2, 5, 50])
```

```
[ ]:
```